

AIM : To write Kotlin program for Hospital Management System.

Program :

```
data class Patient(  
    val id: Int,  
    val name: String,  
    val age: Int,  
    val disease: String  
)
```

```
data class Doctor(  
    val id: Int,  
    val name: String,  
    val specialization: String  
)
```

```
data class Appointment(  
    val patientId: Int,  
    val doctorId: Int  
)
```

```
class Hospital {  
    private val patients = mutableListOf<Patient>()  
    private val doctors = mutableListOf<Doctor>()  
    private val appointments = mutableListOf<Appointment>()  
  
    // Add Patient  
    fun addPatient(id: Int, name: String, age: Int, disease: String) {  
        if (patients.any { it.id == id }) {  
            println("Patient ID already exists.")  
            return  
        }  
    }
```

```

    patients.add(Patient(id, name, age, disease))

    println("Patient added successfully.")
}

// View Patients
fun viewPatients() {
    if (patients.isEmpty()) {
        println("No patients found.")
    } else {
        println("\n--- Patient List ---")
        patients.forEach {
            println("ID: ${it.id} | Name: ${it.name} | Age: ${it.age} | Disease: ${it.disease}")
        }
    }
}

// Add Doctor
fun addDoctor(id: Int, name: String, specialization: String) {
    if (doctors.any { it.id == id }) {
        println("Doctor ID already exists.")
        return
    }
    doctors.add(Doctor(id, name, specialization))
    println("Doctor added successfully.")
}

// View Doctors
fun viewDoctors() {
    if (doctors.isEmpty()) {
        println("No doctors available.")
    } else {

```

```

        println("\n--- Doctor List ---")

        doctors.forEach {
            println("ID: ${it.id} | Name: ${it.name} | Specialization: ${it.specialization}")
        }
    }
}

```

// Book Appointment

```

fun bookAppointment(patientId: Int, doctorId: Int) {
    val patientExists = patients.any { it.id == patientId }
    val doctorExists = doctors.any { it.id == doctorId }

```

```

    if (!patientExists) {
        println("Patient not found.")
        return
    }

```

```

    if (!doctorExists) {
        println("Doctor not found.")
        return
    }

```

```

    appointments.add(Appointment(patientId, doctorId))
    println("Appointment booked successfully.")
}

```

// View Appointments

```

fun viewAppointments() {
    if (appointments.isEmpty()) {
        println("No appointments found.")
    } else {
        println("\n--- Appointments ---")
    }
}

```

```

        appointments.forEach {
            val patient = patients.find { p -> p.id == it.patientId }
            val doctor = doctors.find { d -> d.id == it.doctorId }

            println("Patient: ${patient?.name} → Doctor: ${doctor?.name}")
        }
    }
}

// Remove Patient (Discharge)
fun dischargePatient(id: Int) {
    val removed = patients.removeIf { it.id == id }

    if (removed) {
        appointments.removeIf { it.patientId == id }
        println("Patient discharged successfully.")
    } else {
        println("Patient not found.")
    }
}

fun main() {
    val hospital = Hospital()

    while (true) {
        println("\n===== Hospital Management System =====")
        println("1. Add Patient")
        println("2. View Patients")
        println("3. Add Doctor")
        println("4. View Doctors")
    }
}

```

```
println("5. Book Appointment")
println("6. View Appointments")
println("7. Discharge Patient")
println("8. Exit")
print("Enter choice: ")
```

```
when (readLine()?.toIntOrNull()) {
```

```
1 -> {
```

```
    print("Enter Patient ID: ")
    val id = readLine()?.toIntOrNull()
```

```
    print("Enter Name: ")
    val name = readLine()
```

```
    print("Enter Age: ")
    val age = readLine()?.toIntOrNull()
```

```
    print("Enter Disease: ")
    val disease = readLine()
```

```
    if (id != null && age != null && !name.isNullOrEmpty() && !disease.isNullOrEmpty()) {
        hospital.addPatient(id, name, age, disease)
    } else {
        println("Invalid input.")
    }
}
```

```
2 -> hospital.viewPatients()
```

```
3 -> {
```

```
print("Enter Doctor ID: ")
val id = readLine()?.toIntOrNull()
```

```
print("Enter Name: ")
val name = readLine()
```

```
print("Enter Specialization: ")
val spec = readLine()
```

```
if (id != null && !name.isNullOrBlank() && !spec.isNullOrBlank()) {
    hospital.addDoctor(id, name, spec)
} else {
    println("Invalid input.")
}
}
```

```
4 -> hospital.viewDoctors()
```

```
5 -> {
    print("Enter Patient ID: ")
    val pld = readLine()?.toIntOrNull()

    print("Enter Doctor ID: ")
    val dld = readLine()?.toIntOrNull()

    if (pld != null && dld != null) {
        hospital.bookAppointment(pld, dld)
    } else {
        println("Invalid input.")
    }
}
```

```
6 -> hospital.viewAppointments()
```

```
7 -> {
```

```
    print("Enter Patient ID to discharge: ")
```

```
    val id = readLine()?.toIntOrNull()
```

```
    if (id != null) {
```

```
        hospital.dischargePatient(id)
```

```
    } else {
```

```
        println("Invalid input.")
```

```
    }
```

```
}
```

```
8 -> {
```

```
    println("Exiting system...")
```

```
    return
```

```
}
```

```
else -> println("Invalid choice.")
```

```
}
```

```
}
```

```
}
```

Output :

```
Microsoft Windows [Version 10.0.26200.8117]
(c) Microsoft Corporation. All rights reserved.

C:\Users\densi>cd C:\Users\densi\Downloads\Alwin Hospital

C:\Users\densi\Downloads\Alwin Hospital>kotlinc HospitalManagement.kt

C:\Users\densi\Downloads\Alwin Hospital>Kotlin HospitalManagementKt

===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Assign Doctor
4. Discharge Patient
5. Exit
Enter your choice: 1
Enter Patient ID: 67
Enter Patient Name: Shlok
Enter Age: 20
Enter Disease: Fever
? Patient added successfully!

===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Assign Doctor
4. Discharge Patient
5. Exit
Enter your choice: 3
Enter Patient ID: 67
Enter Doctor Name: Alwin
? Doctor assigned successfully!

===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Assign Doctor
4. Discharge Patient
5. Exit
Enter your choice: 2

--- Patient List ---
ID: 67, Name: Shlok, Age: 20, Disease: Fever, Doctor: Alwin, Status: Admitted
```

```
===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Assign Doctor
4. Discharge Patient
5. Exit
Enter your choice: 4
Enter Patient ID: 67
? Patient discharged successfully!

===== Hospital Management System =====
1. Add Patient
2. View Patients
3. Assign Doctor
4. Discharge Patient
5. Exit
Enter your choice: 5
? Exiting system...

C:\Users\densi\Downloads\Alwin Hospital>
```